KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECTREPORT ON

Bank Account Management System with Deposit, Withdrawal and Transaction History

Submitted by

DHIYA S M

25BCS111

Under theGuidance of

MR.ARAVIND

Technical Trainer Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled **Bank Account Management System with Deposit, Withdrawal and Transaction History** submitted in partial fulfillment of the requirements for the course **[Java Programming]** is a **record of my original work** carried out under the guidance of **[Mr.Aravind]**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.
Ihave acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

DHIYA S.M

CERTIFICATE

This is to certify that the project entitled **Bank Account Management System with Deposit, Withdrawal and Transaction History** is a **bona fide work** carried out by **[DHIYA S.M] [CS-1B-2025]**, [**Computer Science]**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[Java Programming]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore

Date: 10.04.2026

Submitted for the Viva-Voce Examination held on 10.04.2026

Guide:
(Technical Trainer, KGCAS)

MR.ARAVIND

Head of the Department
(Technical Head, KGCAS)

DR.BOOPALAN SUNDARMOORTHY

TABLE OF CONTENTS

ABSTRACT

The Bank Account Management System is a Java-based application developed to manage basic banking operations in an efficient and systematic manner. In traditional banking methods, many tasks such as maintaining account records, calculating balances, and tracking transactions are performed manually, which often leads to errors, delays, and poor record management. To overcome these issues, this project introduces a computerized system that automates banking operations.

This system allows users to create an account, deposit money, withdraw funds, check account balance, and view transaction history. All operations are performed quickly and accurately, reducing manual effort and improving efficiency. The project is developed using Core Java concepts such as classes, objects, methods, loops, and conditional statements. It can be implemented as a console-based application.

The main objective of this system is to provide a simple and user-friendly platform for managing bank accounts while demonstrating the practical use of Java programming in real-world applications.

1. INTRODUCTION

In today's fast-growing digital world, banking systems have evolved from manual processes to automated systems. Earlier, banking operations were performed manually using paper records, which was time-consuming and prone to errors. Maintaining accurate transaction records was difficult, and retrieving information required significant effort.

The Bank Account Management System is designed to overcome these challenges by providing an automated solution. It helps users perform banking operations such as depositing and withdrawing money, checking balance, and maintaining transaction history in an efficient manner. The system ensures accuracy, saves time, and provides a structured way to manage financial data.

This project is developed using Java programming language and demonstrates the use of object-oriented programming concepts. It serves as a basic model for real banking systems and can be further enhanced with advanced features like database connectivity and graphical user interfaces.

1.1 Objective

The main objective of this project is to develop a simple banking system using Java that automates basic banking operations. The system is designed to reduce manual work and improve efficiency.

Key Objectives:

- To perform deposit and withdrawal operations accurately
- To maintain transaction history systematically
- To reduce errors in manual calculations
- To provide a simple and user-friendly interface
- To demonstrate the use of Core Java concepts

1.2 Overview

The Bank Account Management System is a menu-driven application where users can select different options to perform banking operations. When the program starts, it displays a list of options such as account creation, deposit, withdrawal, balance inquiry, and transaction history.

The user selects an option and enters the required details. The system processes the request and displays the result immediately. For example, when a user deposits money, the balance is updated automatically, and the transaction is recorded in the history. Similarly, withdrawal is allowed only if sufficient balance is available.

This system ensures that all operations are performed efficiently and accurately.

1.3 Features

The system provides several important features that make it useful and efficient:

- Account creation for new users
- Deposit money into the account
- Withdraw money with balance checking
- Display current account balance
- Maintain and display transaction history
- Easy-to-use menu-driven interface
- Error handling for invalid operations

2. PROGRAM ANALYSIS

2.1 Problem Definition

In manual banking systems, several problems arise due to the lack of automation. These problems affect efficiency and accuracy.

Common Issues:

- Errors in calculations during transactions
- Difficulty in maintaining proper records
- Time-consuming operations
- Lack of transaction tracking
- Increased chances of data loss
- These issues highlight the need for a computerized banking system.

2.2 Project Overview The Bank Account Management System provides a digital solution to manage banking operations. It simplifies the process of handling accounts by automating all transactions.

The system accepts user input, processes it using logical operations, updates the account balance, and stores transaction details. It ensures that all operations are performed accurately and efficiently. The system also provides a record of all transactions, which helps users track their financial activities.

2.3 Scopes and Limitations Scope:

The project includes managing basic banking operations such as deposit, withdrawal, and transaction tracking. It is mainly designed for educational purposes and serves as a foundation for advanced systems.

Limitations:

- No database (data not stored permanently)
- No advanced security features
- Limited to basic operations

Despite these limitations, the system effectively demonstrates core banking functionality.

3. SYSTEM DESIGN

System design is the stage where the structure and components of the system are planned before the actual implementation. In this phase, the system is divided into different modules, and the workflow of the application is defined. Proper system design helps in organizing the program efficiently and makes development easier.

The Hotel & Restaurant Billing System is designed as a menu-driven application where users can select food items, enter quantities, and generate a final bill automatically.
The system is divided into several modules to perform specific tasks. Each module is responsible for handling a particular function of the system.

3.1 Modules

The system is divided into different modules:

- **Account Creation Module**

  Allows users to create a new account with an initial balance

- **Deposit Module**

  Enables users to add money to their account

- **Withdrawal Module**

  Allows users to withdraw money after checking balance

- **Transaction History Module**

  Records all deposits and withdrawals

- **Balance Inquiry Module**

  Displays the current account balance

Thesemodules work together for smooth functioning.

4. METHODOLOGY

The development of the Bank Account Management System follows a structured and systematic approach. Initially, the problem of manual banking operations was studied and analyzed. Based on this, the system was designed by dividing it into different modules such as deposit, withdrawal, balance inquiry, and transaction history.

The program was implemented using Java programming language by applying object-oriented concepts like classes and methods. After coding, the system was tested with different inputs to ensure correct functionality. Finally, the results were verified and documented. This step-by-step approach helped in developing an efficient and error-free system.

4.1. Problem Analysis

In traditional banking systems, many operations are carried out manually, which leads to several problems. Maintaining records on paper is time-consuming and increases the chances of human errors. Calculating balances manually can result in mistakes, and tracking transaction history becomes difficult.

Additionally, retrieving information from manual records requires more time and effort. There is also a risk of data loss or damage. These issues highlight the need for a computerized system that can automate banking operations, improve accuracy, and save time.

4.2. System Design

The Bank Account Management System is designed as a simple menu-driven application. The system is divided into modules such as account management, deposit, withdrawal, balance inquiry, and transaction history.

Each module performs a specific function. The user interacts with the system through a menu, selects an option, and the system processes the request accordingly. The balance is updated after every transaction, and the details are stored in the transaction history.

The design ensures that all operations are performed efficiently and in an organized manner. It also makes the system easy to understand and maintain.

4.3. Program Development

The program is developed using Java programming language. A class is created to manage account details such as balance and transaction history. Various methods are defined to perform operations like deposit, withdrawal, and displaying account information.

Control structures such as loops and conditional statements are used to handle user input and decision-making. The menu-driven approach allows the program to run continuously until the user chooses to exit.

The development process focuses on simplicity, clarity, and proper implementation of logic to ensure smooth functioning of the system.

4.4. Testing and Validation

The system is tested to ensure that all operations work correctly. Different test cases are used to verify the functionality of deposit, withdrawal, and balance checking.

For example, the system checks whether the withdrawal amount is less than or equal to the available balance. If not, an error message is displayed. Similarly, deposit operations are tested with different amounts to ensure correct balance updates.

Testing helps in identifying and correcting errors, ensuring that the system performs reliably under various conditions.

4.5.Result and Documentation

After successful testing, the results of the system are documented. The system generates the final bill based on the selected food items and quantities.The results are demonstrated through program execution and screenshots showing the input and output of the system.

Proper documentation is prepared to explain the design, implementation, and working of the system.

4.6.Algorithm

1. Start the program
2. Display menu options
3. Accept user choice
4. Perform selected operation (deposit/withdraw)
5. Update account balance
6. Record transaction
7. Display result
8. Repeat until user exits
9. Stop

4.7. Flowchart

Start

↓

Display Menu

↓

Enter Choice

↓

Perform Operation

↓

Update Balance

↓

Check Exit

↓

Back to Menu / End

10. SOURCE CODE

```
import java.util.*;

classBankAccount {
double balance;
ArrayList<String> transactionHistory;

   //Constructor
   BankAccount() {
   balance = 0;
   transactionHistory = new ArrayList<>();
   }

   //Deposit Method
   voiddeposit(double amount) {
      if(amount > 0) {
         balance += amount;
         transactionHistory.add("Deposited: ₹" + amount);
         System.out.println("Amount Deposited Successfully!");
      }else {
         System.out.println("Invalid Amount!");
      }
   }

   //Withdraw Method
   voidwithdraw(double amount) {
      if(amount <= balance && amount > 0) {
         balance -= amount;
```

```
            transactionHistory.add("Withdrawn: ₹" + amount);
            System.out.println("Amount Withdrawn Successfully!");
        } else {
            System.out.println("Insufficient Balance or Invalid Amount!");
        }
    }

    // Display Balance
    void checkBalance() {
        System.out.println("Current Balance: ₹" + balance);
    }

    // Transaction History
    voidshowTransactionHistory() {
        if(transactionHistory.isEmpty()) {
            System.out.println("No Transactions Yet!");
        } else {
            System.out.println("Transaction History:");
            for(Stringt:transactionHistory) {
                System.out.println(t);
            }
        }
    }
}

publicclassBankManagementSystem {
    publicstaticvoidmain(String[] args) {
        Scannersc=newScanner(System.in);
        BankAccountaccount = new BankAccount();
```

```
        intchoice;
        double amount;

        do{
            System.out.println("\n===== BANK ACCOUNT MANAGEMENT SYSTEM
=====");
            System.out.println("1. Deposit");
            System.out.println("2. Withdraw");
            System.out.println("3. Check Balance");
            System.out.println("4. Transaction History");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");

            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    System.out.print("Enter amount to deposit: ₹");
                    amount = sc.nextDouble();
                    account.deposit(amount);
                    break;

                case 2:
                    System.out.print("Enter amount to withdraw: ₹");
                    amount = sc.nextDouble();
                    account.withdraw(amount);
                    break;
```

```
            case 3:
                account.checkBalance();
                break;

            case 4:
                account.showTransactionHistory();
                break;

            case 5:
                System.out.println("Thank you for using the system!");
                break;

            default:
                System.out.println("Invalid Choice! Try again.");
        }

    }while(choice != 5);

    sc.close();
  }
}
```

11. INPUT AND OUTPUT SCREENS

INPUT SCREEN

Enter your choice: 1

Enter amount to deposit: 1000

OUTPUT SCREEN

===== BANK MANAGEMENT SYSTEM =====

1. Deposit

2. Withdraw

3. Balance

4. History

5. Exit

Amount deposited successfully!

Current Balance: 5000

12. DISCUSSION & CONCLUSION

7.1 Discussion

The system was successfully developed using Java. It performs banking operations efficiently and reduces manual effort while ensuring proper record management.

7.2 Conclusion

This project demonstrates how Java can be used to build real-world applications. It helps understand banking operations and object-oriented programming. Future improvements can include database integration, security, and GUI.

13. REFERENCES

The following resources were used during the development of this project:

Books

- Herbert Schildt – Java: The Complete Reference
- Oracle Java Documentation
- GeeksforGeeks
- Javatpoint